

CS 2001 – Data Structures (Fall 2023)

Assignment 2

Topics Covered: ▪ Stack & Queue ADTs	Submission Deadline: <i>Wednesday – October 11, 2023 by 16.00 sharp</i>
Submission Guidelines: ▪ Submit your own original work. Plagiarism will not be tolerated, either done from the internet and/or another student. ▪ Copy all the header (.h) and (.cpp) files in a folder with naming convention given below containing separate subfolders for each question. ▪ Parent Folders should be named as DS_ASS02_XXF_YYYY, where XX represents the batch and YYYY represents the roll number. Compress this folder as DS_ASS01_XXF_YYYY.zip and submit this one file. ☐ Solution of each question should be in a subfolder, named according to the questions. ▪ Any submission not following the submission instructions will not be evaluated.	

You are not allowed to use C++ STL Containers for any purpose. Adopt good structured, modular, and object-oriented programming practices.

Problem # 1:

We want to design a calculator application **Pre-Post Calculator** that evaluates an input Infix expression through Prefix and Postfix notations using the Stack data structure (implemented with linked nodes). An Infix expression may contain positive integers, the five arithmetic operators (+, -, *, /, %), and parenthesis where each token is separated by a space symbol; for example: $24 * (490 + 6 / 12)$. Use the String class functions as necessary to parse the Infix expression.

Refer to your lecture notes for conversion and evaluation algorithms. Provide a menu-driven interface to the user which allows the user to decide how the Infix expression will be evaluated (Prefix notation or Postfix notation). Your application should display all the necessary elements (i.e., converted expression and stack) after every update made during expression conversion and evaluation.

Problem # 2:

Write an application to simulate ‘undo’ and ‘redo’ commands in a usual text editor. We assume that a user will enter commands as strings which will be stored in a stack. Upon the command ‘undo’, the last performed action is undone and removed from the stack. The behavior of ‘redo’ is slightly different from an undo. It will only keep the entries that have been undone through “undo” command, and it also clears the redo history as soon as a new edit command comes in. Your program should also display after each update if the undo and redo commands are enabled or



disabled. Your application should terminate if user enters a 'quit' command. For simplicity, we are just storing the commands; see the below sample program run for better understanding.

```
[undo: disabled, redo: disabled]
Type your command: add text

[undo: enabled, redo: disabled]
Type your command: add text

[undo: enabled, redo: disabled]
Type your command: delete text

[undo: enabled, redo: disabled]
Type your command: undo

delete text undone
[undo: enabled, redo: enabled]

Type your command: color text
[undo: enabled, redo: disabled]

Type your command: quit
Program terminating.
```

Problem # 3:

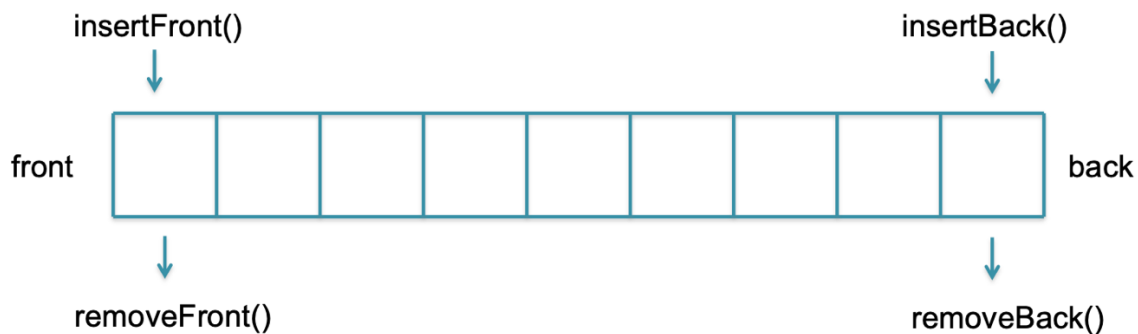
Provide an implementation of **QueueADT using StackADT**, use the following code snippet as a template and implement all the methods. That is, elements of the queueStack class are accessible in a FIFO manner.

```
// Queue implementation using Stacks
class queueStack {
    stack S1, S2;
public:
    queueStack();
    bool isEmpty();
    bool isFull();
    void enqueue (int value);
    int dequeue ();
    ~queueStack();
};
```

```
int main () {
    queueStack Q1;
    Q1.enqueue (178);
    Q1.enqueue (-43);
    Q1.enqueue (953);
    cout<< Q1.dequeue ( );    // 953 will be printed
}
```

Problem # 4:

A **double-ended queue** or **deque** (pronounced as “deck”) is a variant of QueueADT which allows insertions and deletions at both ends using the operations: *insertFront*, *insertBack*, *removeFront*, *removeBack*, as shown in the figure below. If you perform insertions/deletions at one end, it will act as a stack while if you exclusively perform insertions at one end (i.e., back) and deletions at the other end (i.e., front) it will act as a queue. In a simple queue, we increment front and rear indexes upon deletion and insertion, respectively. The same approach will be followed in deque if a user performs insertion/deletion on usual ends. However, as a user inserts a value at front or delete value from rear, the corresponding index should move backward. Provide an array-based C++ implementation for deque that allows insertion and deletion operations in O(1) time. The class template is given at the end for your reference.



Suppose we performed some insertions and deletions at both ends of a deque (of size 10). The deque after the following operations is shown below (grey shaded row shows the array index). Front and rear indexes are set to -1 initially.

deque.insertBack(3);

front = 0, rear = 0

3									
0	1	2	3	4	5	6	7	8	9

deque.insertBack(14);

front = 0, rear = 1

3	14								
0	1	2	3	4	5	6	7	8	9



deque.insertFront (90);

front = 9, rear = 1

3	14								90
0	1	2	3	4	5	6	7	8	9

deque.removeFront ();

front = 0, rear = 1

3	14								
0	1	2	3	4	5	6	7	8	9

deque.removeBack ();

front = 0, rear = 0

3									
0	1	2	3	4	5	6	7	8	9

deque.removeFront ();

front = -1, rear = -1

0	1	2	3	4	5	6	7	8	9

```
// Deque class template
```

```
class deque {  
    int *dequeArr;  
    int maxSize, rear, front;  
public:  
    deque ();  
    bool isEmpty();  
    bool isFull();  
    void insertFront (int value);  
    void insertBack (int value);  
    int removeFront ( );  
    int removeBack ( );  
    ~deque ( );  
};
```